INF1350
Sistemas Reativos

Comunicação MQTT

Prof.: Adriano Branco abranco@inf.puc-rio.br

Slides baseado em aulas anteriores da Prof. Noemi Rodriguez

microcontrolador ESP8266-12F

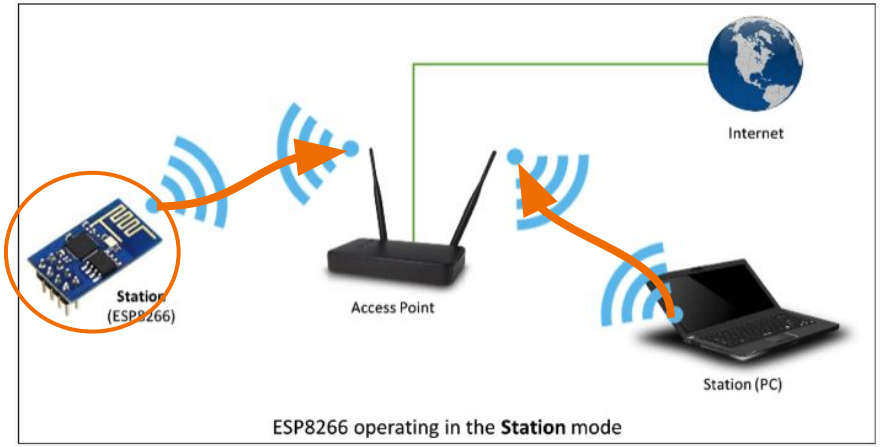
- CPU de 32 bits, 80 MHz
- 64 KB ROM p/ instruções
- 32 KB RAM p/ instruções
- 80 KB RAM p/ dados
- 4MB Fash
- **2.4 GHz wifi - 802.11 b/g/n**
- GPIOs, ADC, PWM
- I^2C, SPI, I^2S, UART





Conexão wifi do NodeMCU

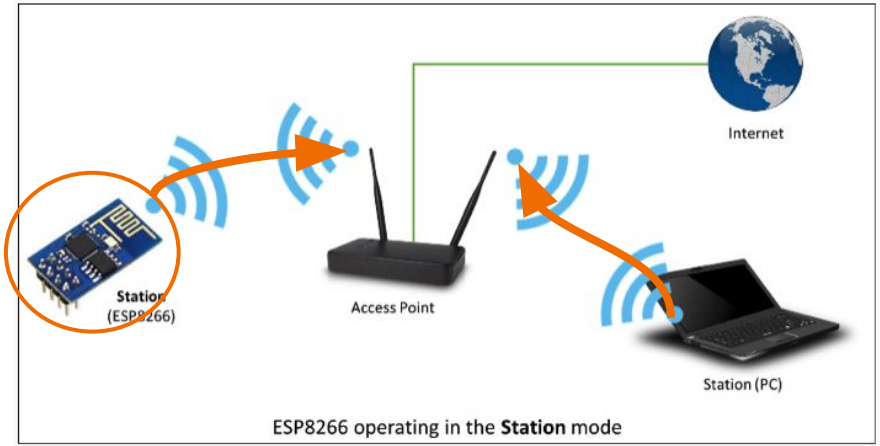
- Microcontrolador oferece suporte para comunicação wifi
- Modos: station e access point
  - **station**: conecta-se em uma rede wifi já estabelecida



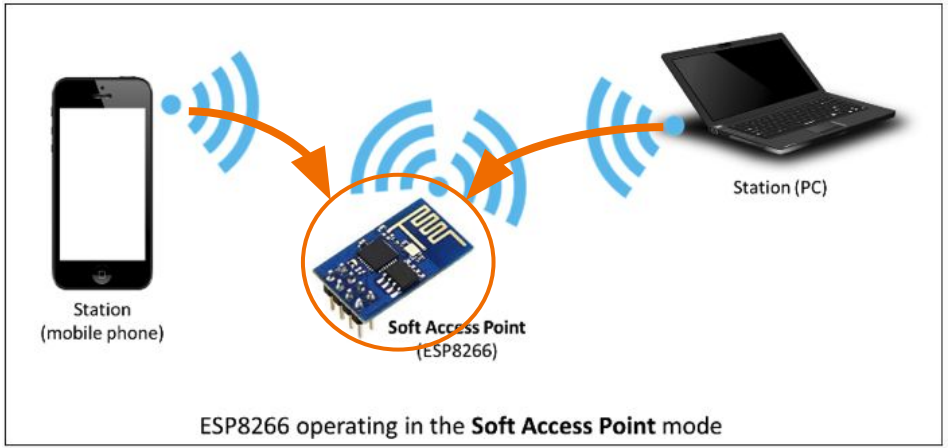
ESP8266 operating in the **Station** mode

Conexão wifi do NodeMCU

- Microcontrolador oferece suporte para comunicação wifi
- Modos: station e access point
  - **station**: conecta-se em uma rede wifi já estabelecida
  - **access point**: cria uma rede wifi a que outros podem se conectar



ESP8266 operating in the **Station** mode



ESP8266 operating in the **Soft Access Point** mode

Conexão wifi do NodeMCU

Estabelecimento de conexão

- Função `wifi.setmode(mode)`
  - mode: `wifi.STATION` (vamos usar principalmente como *station*)
- Função `wifi.sta.config (cfg)`
  - cfg: configuracão como *station*
    - ssid: nome da rede wifi
    - pwd: senha da rede wifi
    - got_ip_cb: callback de conexão
    - save: requisita salvar configuração

Conexão wifi: NodeMCU como *station*

```
-- Conexão: conectando-se a uma rede existente
wificonf = {
  -- verificar ssid e senha
  ssid = "Reativos",
  pwd  = "reativos",
  got_ip_cb = function (con)
                print ("meu IP: ", con.IP)
                -- esperar obter IP para
                -- qualquer comunicação!!!
              end,
   save = false}

wifi.setmode(wifi.STATION)
wifi.sta.config(wificonf)
```

Conexão wifi: NodeMCU como ponto de acesso (AP)

```
-- Conexao: criando rede de wifi nova
wifi.setmode(wifi.SOFTAP)
wifi.ap.config({ssid="nodemcu",pwd="11112222"})
wifi.ap.setip({ip="192.168.0.20",netmask="255.255.255.0",
                    gateway="192.168.0.20"})
print(wifi.ap.getip())
```

e depois de se conectar?

troca de mensagens...

- primitiva básica de comunicação em ambientes distribuídos
- send (destino, mensagem)
- receive (origem, &mensagem)

e depois de se conectar?

troca de mensagens...

- primitiva básica de comunicação em ambientes distribuídos
- send (destino, mensagem)
- receive (origem, &mensagem)

- semântica de envio e recebimento
- endereçamento
- formato de dados

e depois de se conectar?

troca de mensagens...

- primitiva básica de comunicação em ambientes distribuídos
- send (destino, mensagem)
- receive (origem, &mensagem)

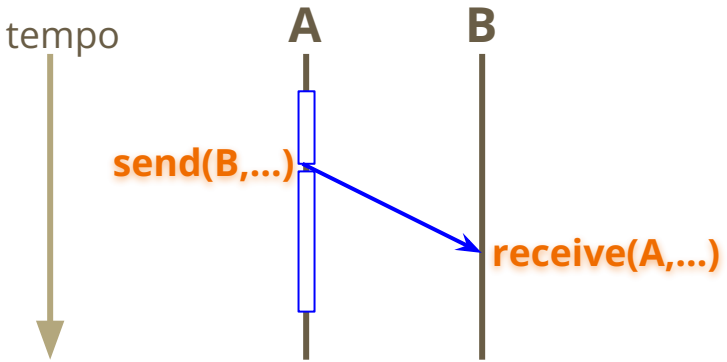
- semântica de envio e recebimento
- endereçamento
- formato de dados

- coordenação entre pares!!!

semântica - envio síncrono e assíncrono

Envio assíncrono: execução procede imediatamente

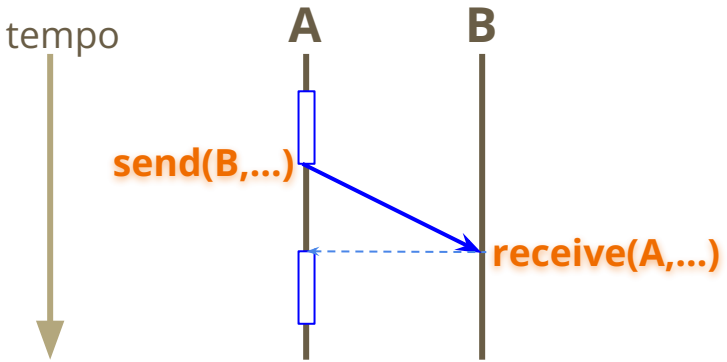
- “bufferização”



semântica - envio síncrono e assíncrono

Envio síncrono: execução procede quando destinatário recebe mensagem

- determinismo



recebimento síncrono e assíncrono

- **recebimento síncrono**
  - é o convencional: execução procede quando há algo a tratar
  - bloqueio: suspensão da linha de execução corrente até chegada da mensagem
  - alternativa de recebimento com timeout
- **recebimento assíncrono**
  - orientação a eventos: chegada de mensagem é encarada como evento a ser tratado
  - recebimento fica implícito

Especificação de destino

- endereçamento direto (acoplamento)
  - endereços e volatilidade
  - serviços de nomes
- caixas de correios e canais (desacoplamento)
  - endereços bem conhecidos
  - comunicação com par “qualquer”

Formato de mensagens

- no nível mais baixo: sequências de bytes
  - interpretação por conta do programa
- apoio de bibliotecas ou linguagens a empacotamento e desempacotamento
  - obj: marshalling e unmarshalling
  - string: formatação
  - bytes: módulo `struct`
- abstrações em vários níveis

API de sockets

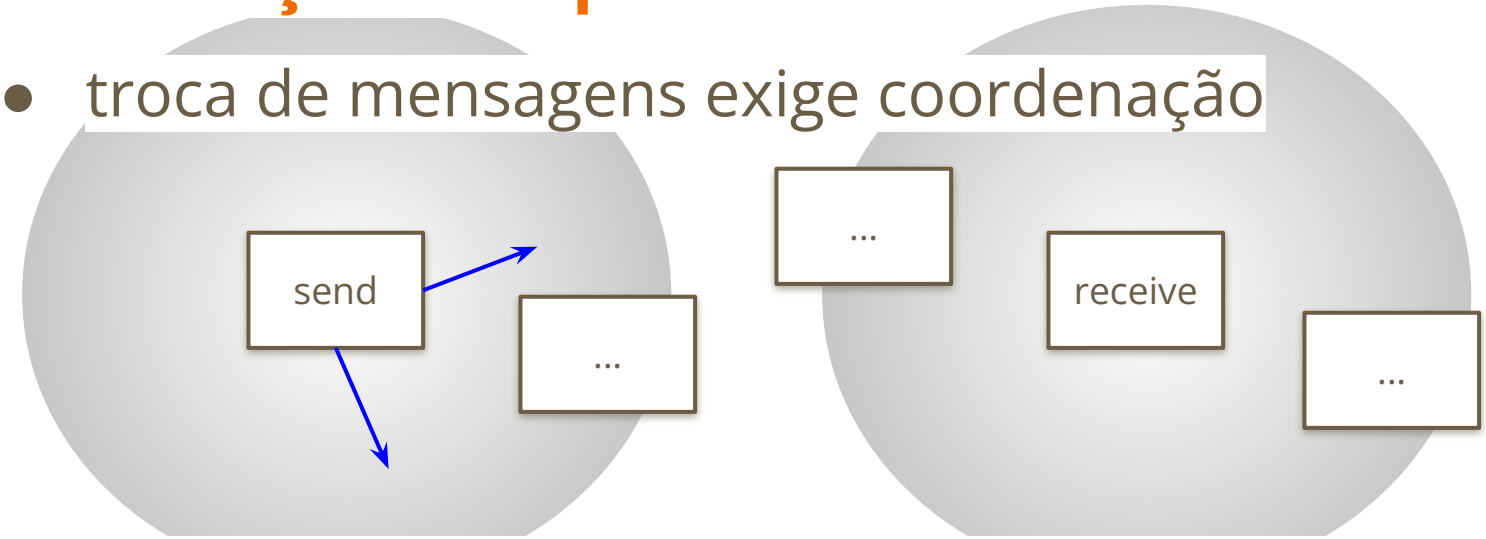
- mensagens são sequência de bytes
- enderecamento por (IP, porta)
- envio (quase sempre) assíncrono e recebimento (quase sempre!!!) síncrono

Falhas parciais

- A noção de “falha parcial” é fundamental nos Sistemas Distribuídos
  - A cooperação entre processos deve ser *tolerante* a falhas individuais
- Não vamos tratar desse assunto aqui!

coordenação entre pares

- troca de mensagens exige coordenação



- quem envia? quem recebe? como é o fluxo de mensagens?
- usos de padrões arquiteturais facilita compreensão

alguns padrões

- cliente-servidor
- pipeline ou filtros
- p2p

desacoplamento: filas e brokers

- intermediário para comunicação
- “terceirização” de garantias

protocolos e serviços

muitos protocolos e serviços

- AMQP (protocolo)
- MQTT (protocolo)
- AWS (serviço)
- Azure Queues (serviço)
- Apache Kafka
- ...

protocolo MQTT

- protocolo binário “leve”
- filas (tópicos) identificados por strings
- mensagens são strings/bytes
- vários *brokers* implementados
- biblioteca para NodeMCU
- servidores de teste públicos:
  - `test.mosquitto.org [1883]`
  - `broker.hivemq.com [1883]`

Em Sistemas Reativos vamos usar o servidor:
`IP: terrasim.inf.puc-rio.br`
`Porta: 1883`

protocolo MQTT - exemplo

```
-- Pode MUDAR meu id para o número da placa!!!!
local meuid = node.chipid()
-- se id cliente já existir vai "expulsar" algum cliente
local m = mqtt.Client("clientid " .. meuid, 120)

function novaInscricao(c)
    -- chamada quando recebe confirmação de inscrição no tópico
end

function conectado(client)
    -- chamada quando consegue se conectar ao broker
    client:subscribe("inf1350-b", 0, novaInscricao)
end

m:connect("terrasim.inf.puc-rio.br", 1883, false, conectado,
            function(client, reason)
                print("failed reason: "..reason)
            end)
```

protocolo MQTT - exemplo

```
-- Pode MUDAR meu id para o número da placa!!!!
local meuid = node.chipid()
local m = mqtt.Client("clientid " .. meuid, 120)

function novaInscricao(c)
    local msgsrec = 0
    function novamsg (c, t, m)
        print ("mensagem ".. msgsrec ..
               ", topico: ".. t .. ", dados: " .. m)
        msgsrec = msgsrec + 1
    end
    c:on("message", novamsg) -- o que faz ao receber mensagem
end

function conectado(client)
    client:subscribe("inf1350-b", 0, novaInscricao)
    -- sempre fica valendo a ultima callback registrada!
end
```

protocolo MQTT - exemplo

```
-- para publicar:
local m = mqtt.Client("clientid " .. meuid, 120)

function publica(c)
    -- aqui chamei quando se conectou mas pode publicar a qq momento
    c:publish("inf1350-a","alo de " .. meuid, 0, 0,
              function(client) print("mandou!") end)
end

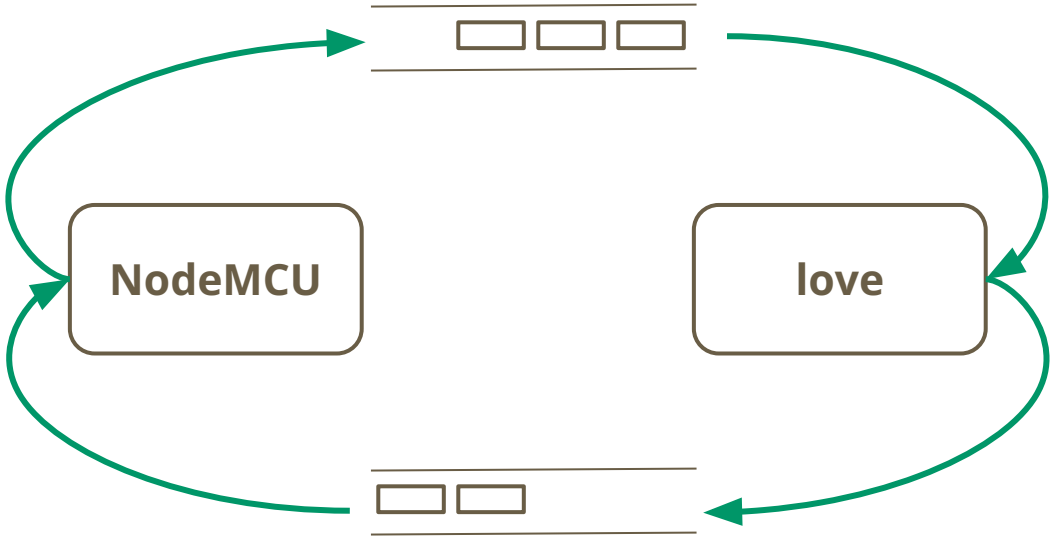
function conectado (client)
    publica(client)
    client:subscribe("inf1350-b", 0, novaInscricao)
end

m:connect("terrasim.inf.puc-rio.br", 1883, false, conectado,
          function(client, reason)
             print("failed reason: "..reason)
          end)
```

MQTT no love

- vamos usar o mqtt para fazer comunicação entre o nodemcu e o love
- no lado love, precisamos usar uma biblioteca (arquivos `mqtt_library.lua` e `utility.lua` podem ficar no mesmo diretório do programa love)
- em `love.load()` fazemos a conexão com o *broker* e em `love.update()` temos que chamar `mqtt_client:handler()`

Comunicação entre NodeMCU e love



MQTT no love - exemplo

```
local function mqttcb (msg)
    print(msg)
    mqtt_client:publish("paranode", "recebi " .. msg)
end

function love.load ()
    ...
    mqtt_client = mqtt.client.create("terrasim.inf.puc-rio.br",1883,mqttcb)
    mqtt_client:connect("cliente love")
    mqtt_client:subscribe({"paralove"})
end

function love.update(dt)
    -- tem que chamar o handler aqui!
    mqtt_client:handler()
end
```

Exercícios

1. Buscar os dois arquivos em https://www.inf.puc-rio.br/~abranco/sr-23.2/code/nodemcu/comm/. Editar `station.lua` com os dados de sua rede. Executar, esperar aparecer o IP, e então executar `testemqtt.lua`. (Esse exercício só fará sentido durante a aula pois nesse intervalo estarei enviando dados para testar esse programa.)
2. Buscar os arquivos em https://www.inf.puc-rio.br/~abranco/sr-23.2/code/lovelua/duaschavesmqtt/ e executar este programa love. Entender o que ele faz.
   a. Escrever um programa para o NodeMCU que se comunique com o programa love e acenda e apague os leds quando clicarmos o mouse nos discos vermelho e verde do programa love.
   b. Estender o programa NodeMCU para enviar mensagens quando as chaves 1 ou a chave 2 forem pressionadas. Estender o programa l□ove para “acender” e “apagar” o disco correspondente quando receber essas mensagens.
   c. Obs: Inclua o ID da etiqueta do seu NodeMCU tanto nos nomes dos tópicos como no nome do ClienteID.